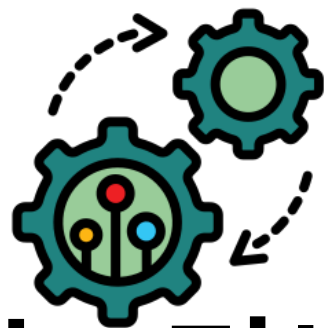




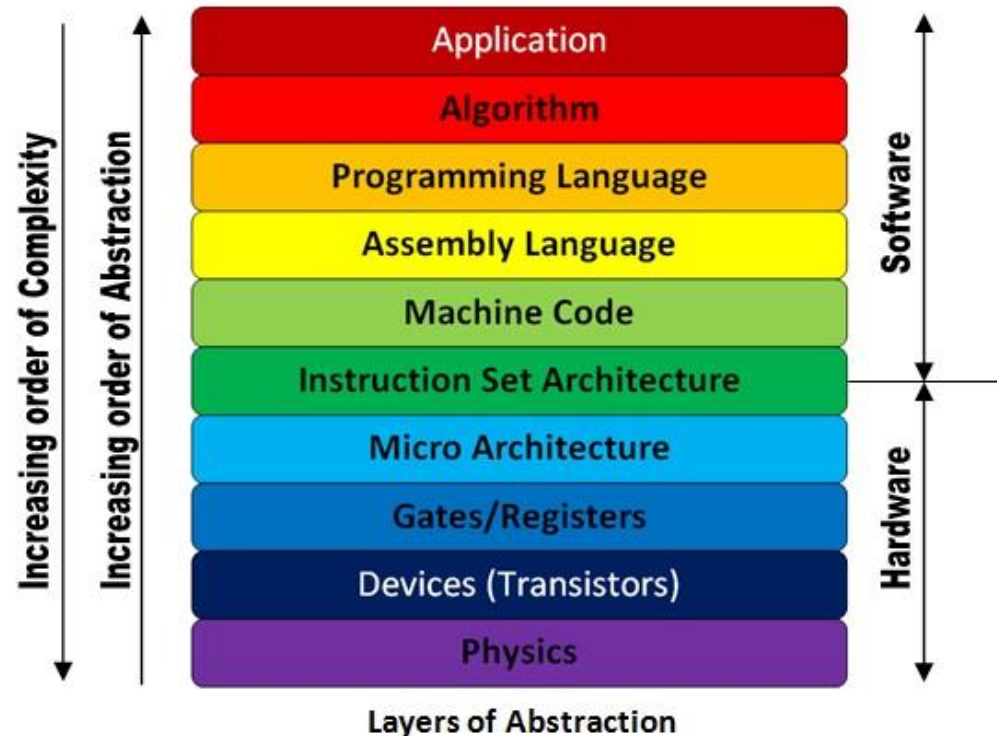
케이녹 리버싱 1회차 과제

17기 최동현 (202511688)

1-1. CPU Architecture & ISA

- ISA (Instruction Set Architecture)

∴ ISA는 소프트웨어(운영체제 및 컴파일러)와 하드웨어(CPU) 간의 **통신 규약(인터페이스)**입니다. CPU가 물리적으로 이해하고 실행할 수 있는 명령어의 집합, 레지스터의 구조, 메모리 접근 방식 등을 정의합니다. 코드를 컴파일하면 타겟 ISA에 종속적인 기계어가 생성됩니다.

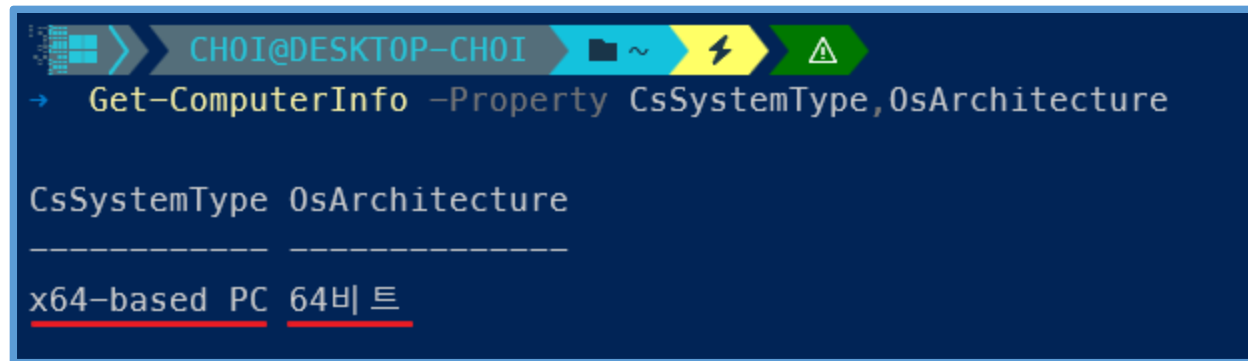


1-2. CPU Architecture & ISA

• CISC vs RISC (x86-64, ARM)

* **CISC (Complex Instruction Set Computer):** 하나의 명령어로 복잡한 연산(메모리 접근 후 산술 연산)을 수행할 수 있도록 설계된 아키텍처입니다. 명령어의 길이가 가변적(1~15바이트)이며, Intel과 AMD의 **x86-64**가 대표적입니다. 전력 소모가 크지만 처리 효율이 높아 데스크탑과 서버 시장을 지배합니다.

* **RISC (Reduced Instruction Set Computer):** 명령어의 기능을 최소화하고 단순화하여 실행 속도를 높인 아키텍처입니다. 명령어의 길이가 고정적이며, 연산을 위해서는 반드시 메모리에서 레지스터로 데이터를 가져오는 과정(Load/Store 구조)이 분리되어 있습니다. 전력 소모가 비교적 적어 Apple Silicon(M1/M2) 및 모바일 기기를 지배하는 **ARM**이 대표적입니다.



```
CHOI@DESKTOP-CHOI ~  
→ Get-ComputerInfo -Property CsSystemType,OsArchitecture  
  
CsSystemType OsArchitecture  
-----  
x64-based PC 64비트
```

2-1. Units of data

- **Binary / Hexadecimal notation**

： 데이터 단위는 CPU와 메모리가 데이터를 처리하는 규격화된 단위입니다. 리버싱에서는 메모리 주소와 데이터 덩어를 분석하기 위해 이 단위를 명확히 인지해야 합니다.

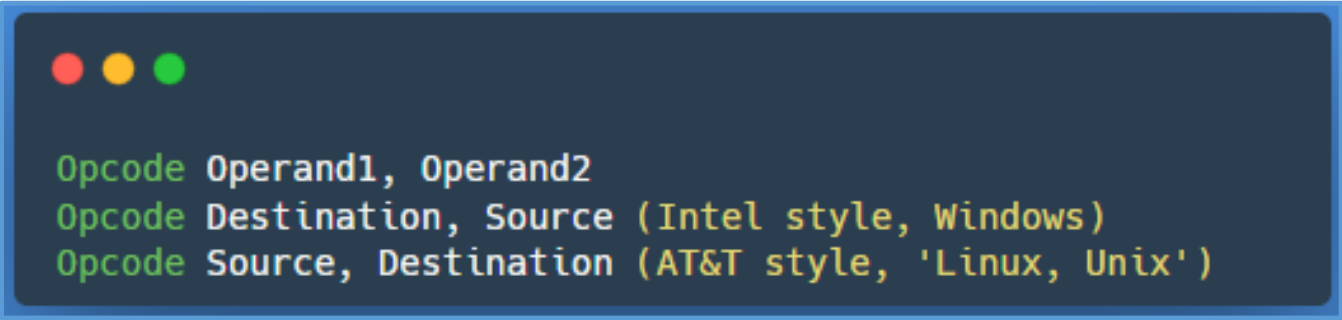
- **Bit (비트):** 0 또는 1을 표현하는 최소 단위.
- **Byte (바이트):** 8 Bits 메모리 주소가 부여되는 최소 단위(Addressable unit)입니다.
- **Word (워드):** x86 아키텍처 역사상 16비트 프로세서 시절의 기준이 굳어져, x86-64에서도 2 Bytes (16 Bits)를 의미합니다.
- **Double Word (DWORD):** 2 Words = 4 Bytes (32 Bits)
- **Quad Word (QWORD):** 4 Words = 8 Bytes (64 Bits) 64비트 시스템의 포인터(메모리 주소) 크기입니다.
- 디버거는 수 많은 0과 1로 이루어진 기계어를 사람이 읽기 편하도록 16진수(Hexadecimal)로 묶어서 표현합니다. (16진수 한 자리는 4비트(1111, 0 ~ 15)를 표현하므로, 2자리가 1바이트가 됩니다. 예: 0xFF = 1바이트)

3-1. Assembly Lang & Machine Code

- 어셈블리어와 기계어의 관계 :

- 기계어 (Machine Code): CPU가 직접 디코딩하고 실행하는 0과 1의 바이너리 데이터입니다. (48 89 e5)
- 어셈블리어 (Assembly Language): 난해한 기계어를 사람이 읽을 수 있는 니모닉(Mnemonic 기호)으로 1:1 매칭시킨 저수준 언어입니다. 디스어셈블러(Disassembler)가 기계어를 어셈블리어로 역변환합니다.

어셈블리 명령어의 기본 구조 : 명령어는 Opcode(연산 코드)와 Operand(피연산자)로 구성됩니다.



```
Opcode Operand1, Operand2
Opcode Destination, Source (Intel style, Windows)
Opcode Source, Destination (AT&T style, 'Linux, Unix')
```

3-2. Assembly Lang & Machine Code

• 디스어셈블(Disassemble)

： 컴퓨터가 실행할 수 있는 형태인 기계어(바이너리)를 사람이 읽을 수 있는 어셈블리어로 변환하는 역과정

```
(root@kali)-[~]
# objdump -d -M Intel /bin/ls

/bin/ls:      file format elf64-x86-64      elf = 리눅스 시스템 바이너리 형식

Disassembly of section .init:

0000000000004000 <.init>:
4000:  48 83 ec 08      sub    $0x8,%rsp
4004:  48 8b 05 bd 2f 02 00  mov    0x22fbd(%rip),%rax      # 26fc8 <__gmon_start__@Base>
400b:  48 85 c0          test   %rax,%rax
400e:  74 02            je     4012 <__ctype_toupper_loc@plt-0x1e>
4010:  ff d0           call   *%rax
4012:  48 83 c4 08      add    $0x8,%rsp
4016:  c3              ret
```

4-1. x86-64 Register

• x86-64 주요 레지스터의 종류와 역할

： 레지스터(Register)는 CPU 내부에 존재하는 초고속 메모리 공간입니다. 메모리(RAM)에 접근하는 것보다 수백 배 빠르므로 모든 연산은 레지스터 위에서 이루어집니다. 64비트 아키텍처의 범용 레지스터는 모두 64비트(8바이트, QWORD) 크기를 가집니다.

- 범용 레지스터 (GPR, General Purpose Register):

- RAX (Accumulator): 산술 연산의 결과나, 함수 호출이 종료된 후의 반환값(Return Value)이 저장됩니다.
- RBX (Base): 메모리 주소 지정 시 베이스 레지스터로 주로 쓰입니다.
- RCX (Counter): 반복문(Loop) 처리 시 반복 횟수 카운터로 사용됩니다.
- RDX (Data): 곱셈/나눗셈 등 데이터 연산 시 RAX의 보조 레지스터로 쓰입니다.

- 특수 목적 레지스터:

- RSP (Stack Pointer): 현재 스택의 최상단(가장 낮은 주소)을 가리킵니다.
- RBP (Base Pointer): 현재 스택 프레임의 기준점(바닥)을 가리킵니다.
- RIP (Instruction Pointer): CPU가 다음에 실행할 기계어 명령어의 메모리 주소를 가리키는 가장 핵심적인 레지스터입니다.
- RFLAGS: 비교 연산 등의 결과 상태(Zero Flag, Sign Flag 등)를 비트 단위로 저장합니다.

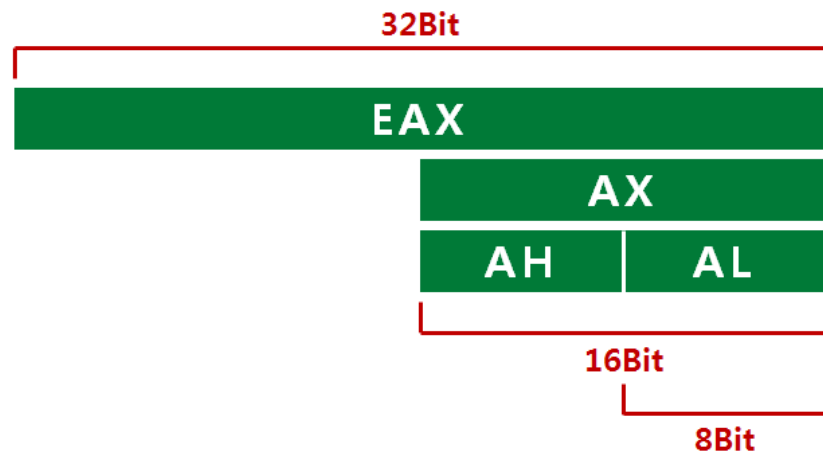
4-2. x86-64 Register

• x86-64 주요 레지스터의 종류와 역할

∴ x86-64 아키텍처는 과거 32비트 아키텍처(IA-32)와의 하위 호환성을 유지합니다.

R 레지스터와 E 레지스터의 차이 분석:

- RAX는 전체 64비트 공간을 의미합니다.
- EAX (**Extended**)는 RAX의 하위 32비트(4바이트)만을 지칭합니다.
- 동일하게 AX는 하위 16비트, AL은 하위 8비트를 지칭합니다. 즉, 같은 물리적 메모리 소자를 크기에 따라 다르게 부르는 것입니다. EAX를 조작하면 RAX의 하위 32비트가 바뀝니다.



5-1. Assembly Commands

• 주요 어셈블리 명령어의 기능 (#1)

1) 데이터 전송 및 주소 계산

- **mov dst, src** : src의 데이터를 dst로 복사합니다.
 - **mov rax, 1** (rax에 1 대입) / **mov rax, QWORD PTR [rbx]** (rbx 주소의 데이터를 역참조하여 8바이트를 rax에 복사)
- **lea dst, [src]** : Load Effective Address. 대괄호 [] 내부의 연산을 주소값 그 자체로 계산하여 dst에 담습니다. 데이터 역참조가 발생하지 않습니다.
 - **lea rax, [rbx+8]** (rbx 값에 8을 더한 주소값을 rax에 대입)

2) 산술 연산

- **add dst, src / sub dst, src** : 덧셈 및 뺄셈 연산. 결과를 dst에 덮어씁니다.

3) 논리 비교 연산 (RFLAGS 레지스터 변경)

- **cmp op1, op2** : op1에서 op2를 빼는 연산을 내부적으로 수행하여 결과를 버리고 RFLAGS(Zero Flag)만 갱신합니다.
- **test op1, op2** : op1과 op2의 비트 AND 연산을 내부적으로 수행하여 RFLAGS를 갱신합니다. 주로 레지스터가 0인지 확인할 때 씁니다. (test eax, eax)
- **xor op1, op2** : 비트 배타적 논리합 연산. 두 값이 같으면 0이 되므로, 레지스터를 0으로 초기화할 때 **mov rax, 0**보다 코드가 짧은 **xor rax, rax**를 자주 씁니다.

5-2. Assembly Commands

• 주요 어셈블리 명령어의 기능 (#2)

4) 분기 명령어 (Branch)

- **jmp** 주소 : 무조건 점프합니다. (RIP 변경)
- **je / jz** : Jump if Equal / Jump if Zero. 직전 **cmp** 결과가 같거나(Zero Flag=1), **test** 결과가 0이면 점프합니다.
- **jne / jnz** : 다르면(또는 0이 아니면) 점프합니다.

5) 함수 제어

- **call** 주소 : 함수를 호출합니다. 내부적으로 현재 RIP(다음 명령어 주소)를 스택에 push하고, 타겟 주소로 jmp 합니다.
- **Ret** : 함수를 종료합니다. 내부적으로 스택에서 값을 pop하여 RIP에 대입함으로써 원래 실행 흐름으로 되돌아갑니다.

6-1. Exercism

- Hello World

```
hello_world.asm
1  default rel
2
3  section .rodata
4  msg: db "Hello, World!", 0
5
6  section .text
7  global hello
8  hello:
9      lea rax, [msg]
10     ret
11
12 %ifidn __OUTPUT_FORMAT__,elf64
13 section .note.GNU-stack noalloc noexec nowrite progbits
14 %endif
15
```

6-2. Exercism

• Lasagna

```
3  %include "debug.mac"
4
5  EXPECTED_OVEN_TIME equ 40      ; 예상되는 총 오븐 조리 시간 (40분)
6  MINUTES_PER_LAYER equ 2       ; 라자냐 1층을 준비하는데 걸리는 시간 (2분)
7
8  section .text
9  ; You should implement functions in the .text section
10 ; the global directive makes a function visible to the test files
11 global expected_minutes_in_oven
12 expected_minutes_in_oven:
13     mov rax, EXPECTED_OVEN_TIME ; 반환 레지스터인 rax에 상수 40을 넣는다
14     ret
15
16 global remaining_minutes_in_oven
17 remaining_minutes_in_oven:
18     mov rax, EXPECTED_OVEN_TIME ; rax에 먼저 40을 넣는다 (rax = 40)
19     sub rax, rdi ; rax에서 rdi(실제 시간)를 뺀다 (rax = 40 - rdi)
20     ret
21
22 global preparation_time_in_minutes
23 preparation_time_in_minutes:
24     mov rax, rdi ; 계산을 위해 rax에 rdi(층 수) 값을 복사한다
25     imul rax, MINUTES_PER_LAYER ; rax에 2를 곱한다 (rax = 층 수 * 2)
26     ret
```

```
28 global elapsed_time_in_minutes
29 elapsed_time_in_minutes:
30     mov rax, rdi ; rax에 rdi(층 수)를 복사한다
31     imul rax, MINUTES_PER_LAYER ; rax에 2를 곱해 준비 시간을 계산한다 (rax = 층 수 * 2)
32     add rax, rsi ; 계산된 준비 시간에 rsi(오븐 시간)를 더한다 (rax = 준비 시간 + 오븐 시간)
33     ret
34
35 %ifidn __OUTPUT_FORMAT__,elf64
36 section .note.GNU-stack noalloc noexec nowrite progbits
37 %endif
38
```



감사합니다

17기 최동현